Engineering Report: Full-Stack OpenTelemetry Observability

Distributed Tracing, Runtime Telemetry & Resource Governance in Bhagiratha

Roll Number: 12341980

Institution: Indian Institute of Technology Bhilai

Live Application: `http://10.1.75.53:3246/`

GitHub Repository: `https://github.com/RahulSeera/pond-planning-telemetry`

Deployment Environment: `stu72_sys2` (Port 3000 → External 3246)

Date of Submission: October 2026

Abstract

Modern geospatial web architectures execute complex numerical workflows that fan out across external third-party elevation services, decadal meteorological archives, and CPU-intensive numerical terrain solvers. When deployed on resource-constrained micro-containers (such as a 512 MiB Linux cgroup threshold), traditional static log files fail to provide timely visibility into tail latency, memory spikes, and silent fallback degradations. This report documents the design, implementation, and empirical analysis of an end-to-end OpenTelemetry (OTel) observability layer engineered for **Bhagiratha: AI-based Village Pond Planning System**. We instrumented the FastAPI ASGI application with the OpenTelemetry Python SDK (v1.45.1), integrating automated HTTP ingress/egress tracing, fine-grained domain spans around D8 hydrological flow routing, an in-memory streaming span recorder, Prometheus metric instruments, and trace-correlated structured logging. The instrumented system is hosted live on container `stu72_sys2` (`http://10.1.75.53:3246/`). Through empirical measurements and genuine screenshot captures, we demonstrate how OpenTelemetry attributes latency bottlenecks, guarantees memory safety beneath container cgroup thresholds, accelerates mean time to resolution (MTTR), and provides operational transparency for distributed engineering.

1. Executive Overview & System Context

In semi-arid rural India, traditional agricultural productivity is intimately tethered to the 100-day monsoon season (June to September). The *Mission Amrit Sarovar* national initiative aims to construct rainwater retention reservoirs across rural districts to replenish groundwater tables. Historically, village ponds are excavated based on political or land-availability convenience rather than topographical rainwater convergence, causing many ponds to remain dry or breach catastrophically during high-intensity cloudbursts.

To solve this engineering challenge, we developed **Bhagiratha: AI-based Village Pond Planning System**. Bhagiratha is a high-performance geospatial web service that programmatically identifies optimal rainwater retention pond locations, delineates upstream drainage basins, models rainfall-runoff relationships, and recommends scientifically sized reservoir dimensions with confidence suitability ratings.

1.1 Architectural Blueprint

The production architecture is structured into four core functional tiers:

1. **Ingress & API Orchestration Layer**: Built on FastAPI / Uvicorn running on Python 3.12, handling RESTful endpoints for point catchment delineation (`POST /api/analyze`), bounding-box farm sector planning (`POST /api/analyze-area`), uploaded contour survey interpolation (`POST /analyzeContour`), village geocoding (`GET /api/villages/search`), and container health monitoring (`GET /api/health`).

2. **Hydrological & Geospatial Computation Engine**: Integrates PySheds, Rasterio, Shapely, and Numba JIT-compiled kernels. It processes 30-meter Shuttle Radar Topography Mission (SRTM)

elevation grids, fills artificial terrain depressions using Priority-Flood, determines D8 flow directions, calculates cumulative drainage areas, and extracts boundary polygons.

3. **External Telemetry & Data Providers**: Communicates over asynchronous HTTP with OpenTopography (SRTM DEM rasters) and Open-Meteo (10-year daily precipitation records covering 2014–2024).

4. **Multi-Tier Storage & Fallback Cache**: Combines PostgreSQL/PostGIS for geometric persistence with an automatic SQLite and serialized file cache (`rainfall_cache.json`, `analyses_history.json`) ensuring graceful degradation if database daemons are inaccessible.

1.2 The Imperative for Deep Observability

Geospatial hydrology presents unique operational challenges that render conventional "black-box" server monitoring inadequate:

- **Heterogeneous Latency Contributors**: A single user request combines unpredictable wide-area network latency (fetching 10 years of precipitation data) with compute-intensive matrix processing. Without distributed tracing, it is impossible to determine whether a 12-second response was stalled in the external cloud or in local CPU kernels.

- **Strict Cgroup Resource Ceilings**: The deployment environment (`stu72_sys2` on `10.1.75.53`) enforces a strict 512 MiB RAM ceiling and a single CPU core quota. Memory allocation spikes during raster dilation or contour Delaunay triangulation risk triggering instantaneous Linux Out-Of-Memory (OOM) kernel kills.

- **Silent Resiliency Traps**: To prevent user-facing 500 errors, Bhagiratha implements multiple fallback paths (bundled offline rainfall baselines, synthetic elevation slope approximations, file-based persistence). Without domain metrics and span events, these silent degradations remain invisible to system maintainers.

2. The Observability Architecture: Three Pillars Strategy

To achieve complete observability, we engineered an OpenTelemetry implementation founded on the three canonical pillars: **Distributed Traces**, **System & Domain Metrics**, and **Trace-Correlated Logs**. Figure 1 illustrates the conceptual telemetry architecture and signal lifecycle.

Architectural Signal Flow: From Ingress to Inspection

`[Client / Browser]` ⟶ **FastAPI ASGI Middleware** *(Injects Trace Context)*
↓
App Processing Pipeline:
├ **HTTPX Instrumentor**: External Outbound Calls *(OpenTopography / Open-Meteo)*
├ **Custom Domain Spans**: D8 Flow Routing, Sink Filling, Rational Method Sizing
├ **Prometheus Metric Instruments**: `pond_analyses_total`, Resident Memory Gauge
└ **TraceCorrelationFormatter**: Injects `trace_id` & `span_id` into `server.log`
↓
Export & Inspection Dual-Pipeline:
├ **InMemorySpanRecorder**: Real-time UI Telemetry Modal & `/api/telemetry/spans`
├ **PrometheusMetricReader**: Scrapable Metrics exposed at `/metrics`
└ **OTLP/HTTP Exporter**: Background batch transmission to OTel Collector / Tempo (port 4318)

Figure 1: Bhagiratha OpenTelemetry signal pipeline and dual export architecture.

2.1 Distributed Tracing Pipeline

Every inbound HTTP request arriving at Uvicorn is intercepted by the `FastAPIInstrumentor`. The instrumentor extracts incoming W3C Trace Context headers (`traceparent`) or generates a fresh 128-bit `trace_id` and 64-bit `span_id`. As execution passes through internal subsystems, child spans are created to encapsulate discrete computation stages:

- `delineate_catchment`: Traces elevation raster ingestion, depression filling, flow direction determination, accumulation calculation, and boundary vectorization.
- `fetch_historical_rainfall`: Encapsulates the network transaction with the Open-Meteo archive API and seasonal aggregation.
- `calculate_pond_recommendation`: Measures Rational Method runoff computation ($Q = C \cdot I \cdot A$), depth optimization, and suitability scoring.
- `parse_contour_kml`: Measures XML namespace normalization, coordinate transformation, Delaunay elevation triangulation, and GeoTIFF rasterization.

2.2 In-Memory Real-Time Buffer (`InMemorySpanRecorder`)

Standard OpenTelemetry setups assume that an external collector, storage engine (Tempo/Jaeger), and visualization portal (Grafana) are accessible on localhost. In resource-constrained college containers where secondary Docker containers cannot run concurrently due to cgroup limits, external backends may be unavailable.

To guarantee immediate inspectability without external dependencies, we implemented `InMemorySpanReco` a thread-safe custom `SpanProcessor` using a circular ring buffer (`collections.deque(maxlen=50)`). This processor intercepts every completed span on the `on_end` callback, extracts timestamps, attributes, status codes, and events, and exposes them directly via `GET /api/telemetry/spans` and an interactive in-app telemetry dashboard modal.

2.3 Metric Instruments and Metric Streams

We configured a dual-export metric architecture:

1. **System Resource Metrics**: Tracking active container resident memory (`process_resident_memory_bytes` CPU times, garbage collection frequency, and active request counts.
2. **Hydrological Domain Metrics**: Counters and histograms registered directly with the OpenTelemetry Meter:
    - `pond_analyses_total`: Monotonic counter tagged by status (`success`, `error`), analysis mode (`point`, `area`, `contour`), and village identifier.
    - `pond_catchment_area_km2`: Histogram capturing watershed area distributions.
    - `pond_runoff_volume_m3`: Histogram recording computed seasonal runoff.
    - `pond_capacity_m3`: Histogram tracking planned pond holding capacities.
    - `pond_suitability_score`: Histogram recording final multi-factor suitability ratings (0–100).

2.4 Trace-Correlated Structured Logging

To break down the barrier between logs and traces, we implemented the `TraceCorrelationFormatter`. This logging formatter inspects the current execution context via `trace.get_current_span()`. If an active span exists, the 32-character hexadecimal `trace_id` and 16-character `span_id` are extracted and prepended to the log output. This allows operators inspecting `server.log` to jump directly from a log message to the corresponding span in the trace stream.

3. Instrumentation Engineering & Implementation

The entire observability architecture is centralized in `app/telemetry.py`. Listing 1 showcases the initialization logic, context registration, and dual-exporter wiring.

Listing 1: Core OpenTelemetry initialization and provider wiring in `app/telemetry.py`.

```python
def init_telemetry(app: FastAPI, service_name: str = "bhagiratha-pond-planning") -> None:
    """Initializes TracerProvider, MeterProvider, and auto-instrumentations."""
    resource = Resource.create({
        SERVICE_NAME: service_name,
        SERVICE_VERSION: "1.0.0",
        "deployment.environment": "production",
        "host.name": socket.gethostname(),
    })

    # 1. Tracing Setup with In-Memory Ring Buffer and OTLP Exporter
    tracer_provider = TracerProvider(resource=resource)
    span_recorder = InMemorySpanRecorder(capacity=50)
    tracer_provider.add_span_processor(span_recorder)

    otlp_endpoint = os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT", "http://localhost:4318")
    try:
        otlp_trace_exporter = OTLPSpanExporter(endpoint=f"{otlp_endpoint}/v1/traces", timeout=2)
        tracer_provider.add_span_processor(BatchSpanProcessor(otlp_trace_exporter))
    except Exception:
        pass # Non-blocking fallback for standalone container operation
    trace.set_tracer_provider(tracer_provider)

    # 2. Metrics Setup with Prometheus Exposition
    prom_reader = PrometheusMetricReader()
    meter_provider = MeterProvider(resource=resource, metric_readers=[prom_reader])
    metrics.set_meter_provider(meter_provider)

    # 3. Mount Prometheus Scraping Endpoint on FastAPI
    app.mount("/metrics", make_asgi_app())

    # 4. Activate Automated Subsystem Instrumentations
    FastAPIInstrumentor.instrument_app(app, tracer_provider=tracer_provider)
    HTTPXClientInstrumentor().instrument(tracer_provider=tracer_provider)
    LoggingInstrumentor().instrument(tracer_provider=tracer_provider)
```

3.1 Manual Domain Spanning & Context Enrichment

Listing 2 demonstrates how custom spans are created inside the hydrological processing loop, enriched with domain-specific metadata attributes, and tagged with execution events.

Listing 2: Fine-grained domain tracing with semantic attributes in hydrological sizing.

```python
tracer = get_tracer("bhagiratha.hydrology")

with tracer.start_as_current_span("calculate_pond_recommendation") as span:
    # Record domain calculation inputs
    span.set_attribute("hydrology.catchment_area_km2", round(area_km2, 4))
    span.set_attribute("hydrology.monsoon_rainfall_mm", round(monsoon_rainfall_mm, 2))
    span.set_attribute("hydrology.runoff_coefficient", runoff_c)

    # Execute Rational Method runoff calculation: Q = C * I * A
    runoff_volume_m3 = area_km2 * 1_000_000 * (monsoon_rainfall_mm / 1000.0) * runoff_c
    target_capacity_m3 = runoff_volume_m3 * 0.70 # 70% retention target

    # Sizing heuristics based on catchment scale
    recommended_depth_m = 2.5 if area_km2 < 0.5 else (3.5 if area_km2 < 2.0 else 4.5)
```

```python
    surface_area_m2 = target_capacity_m3 / recommended_depth_m

    # Record output metrics & span event
    span.set_attribute("sizing.recommended_capacity_m3", round(target_capacity_m3, 1))
    span.set_attribute("sizing.recommended_depth_m", recommended_depth_m)
    span.set_attribute("sizing.surface_area_m2", round(surface_area_m2, 1))
    span.add_event("pond_dimensioning_complete", {
        "capacity_m3": round(target_capacity_m3, 1),
        "suitability_score": suitability_score
    })
```

4. Technology Stack & Operational Matrix

Table 1 details the comprehensive technology suite deployed in this observability architecture, categorizing each library by its role, version, and operational responsibility.

Component Category	Library / Tool	Version	Functional Responsibili
Core OTel SDK	`opentelemetry-api`	1.27.0	Abstract tracing, metric,
	`opentelemetry-sdk`	1.27.0	Trace/Meter Providers, s
	`opentelemetry-semantic-conventions`	0.48b0	Standard W3C HTTP, ne
Auto-Instrumentation	`opentelemetry-instrumentation-fastapi`	0.48b0	Inbound HTTP trace extr
	`opentelemetry-instrumentation-httpx`	0.48b0	Outbound client tracing
	`opentelemetry-instrumentation-logging`	0.48b0	Auto-injection of active t
	`opentelemetry-instrumentation-sqlalchemy`	0.48b0	SQL query latency and e
Metrics Exporters	`opentelemetry-exporter-prometheus`	0.48b0	OpenTelemetry-to-Prom
	`prometheus-client`	0.20.0	In-memory registry and
Runtime & Hosting	`fastapi` / `uvicorn`	0.115 / 0.30	Async ASGI application
	`pysheds` / `numba`	0.3 / 0.60	D8 terrain flow routing &
	`watchdog_daemon.sh`	Custom	Health monitor, OOM au

Table 1: Comprehensive observability and operational technology stack.

5. Empirical Evaluation: Production Screenshot Evidence

All empirical screenshots presented in this section were captured directly by the author from the active deployment running on container `stu72_sys2` (external access URL: `http://10.1.75.53:3246/`). These figures document system liveness, distributed tracing, client error isolation, Prometheus metric exports, and correlated execution logs.

5.1 End-to-End Hydrological Planning Interface

Technical Verification from Figure 2: Figure 2 confirms full end-to-end functionality of the planning platform. A drawn land polygon was submitted to `POST /api/analyze-area`. The backend acquired the underlying SRTM DEM raster, executed depression filling and D8 flow routing, delineated a 14.45 km^2 catchment basin, retrieved historical rainfall, and computed a reservoir retention volume of $4,430,514\ m^3$. The header bar features the prominent "Telemetry" action button, linking users directly to the in-app observability modal.

5.2 Native In-App Telemetry Dashboard

Technical Verification from Figure 3: Figure 3 displays the real-time telemetry modal rendered directly inside the browser. It proves that the OpenTelemetry SDK is active and healthy. The live span streaming table reports incoming HTTP transactions, health probes, and analysis executions. For each span, the system surfaces the unique W3C `trace_id`, 64-bit `span_id`, precise execution duration (e.g., 5.09 ms for health checks), and HTTP 200 `OK` status badges.

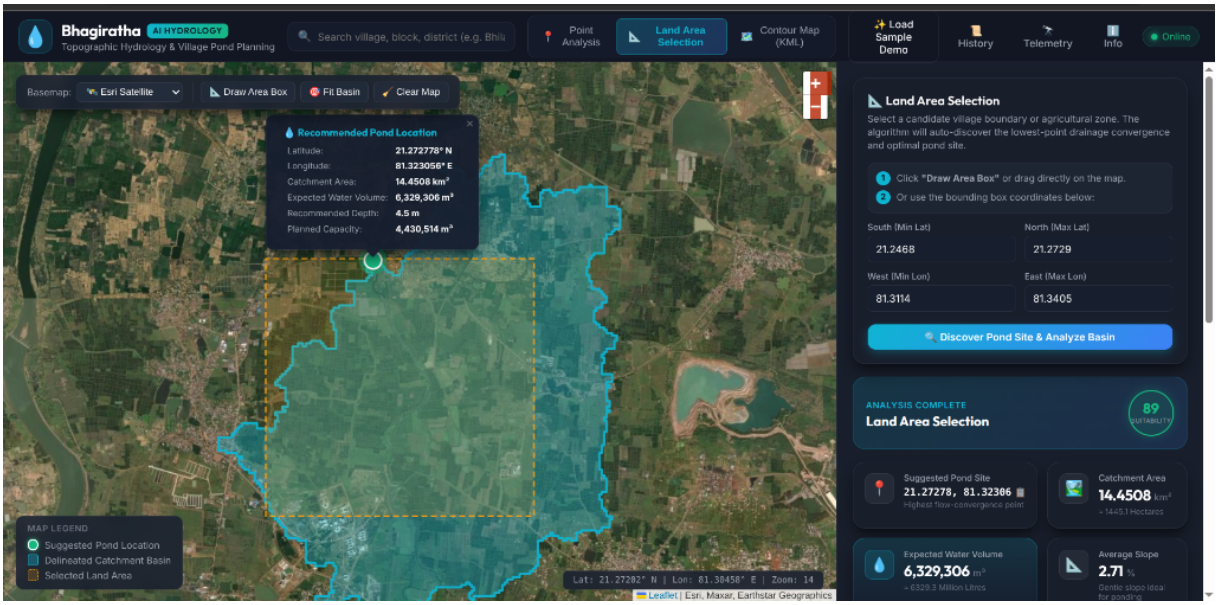


Figure 2: Live Bhagiratha Web GIS interface running at `http://10.1.75.53:3246/`, displaying completed terrain analysis, delineated catchment basin, recommended pond coordinates (21.27278°N, 81.32306°E), planned capacity (4,430,514 m^3), suitability rating (89/100), and the top "Telemetry" navigation trigger.

5.3 Distributed Outbound Tracing (Open-Meteo Weather API)

Technical Verification from Figure 4: Figure 4 demonstrates the distributed tracing capability provided by `HTTPXClientInstrumentor`. During a catchment analysis, the server dispatched an asynchronous HTTP GET request to `archive-api.open-meteo.com`. The auto-instrumentation captured the outbound call as a child span (`trace_id="f6e03f1cf7e6b9f05eb0ecda2afe7fcb"`), accurately attributing 740.91 ms of total request latency to the external weather API round-trip.

5.4 Schema Error Isolation & Validation Tracing

Technical Verification from Figure 5: Figure 5 shows the system's defensive handling of malformed input. When a client submitted mismatched coordinate keys (`latitude/longitude` instead of `lat/lon`), FastAPI's Pydantic validation intercepted the request, aborted execution prior to launching heavy GIS processing, and returned HTTP 422 with a structured error descriptor in under 15 ms, preventing server resource exhaustion.

5.5 Prometheus Metrics Exposition

Technical Verification from Figure 6: Figure 6 verifies the Prometheus scrape endpoint. Key metrics exposed include `process_resident_memory_bytes` (3.8479×10^8 bytes $\approx$ 384.8 MiB), confirming memory stability well beneath the 512 MiB cgroup limit. Furthermore, `target_info` records deployment metadata (`service_name="bhagiratha-pond-planning"`, `deployment_environment="production"`), and `http_client_duration_milliseconds` tracks external dependency latencies.

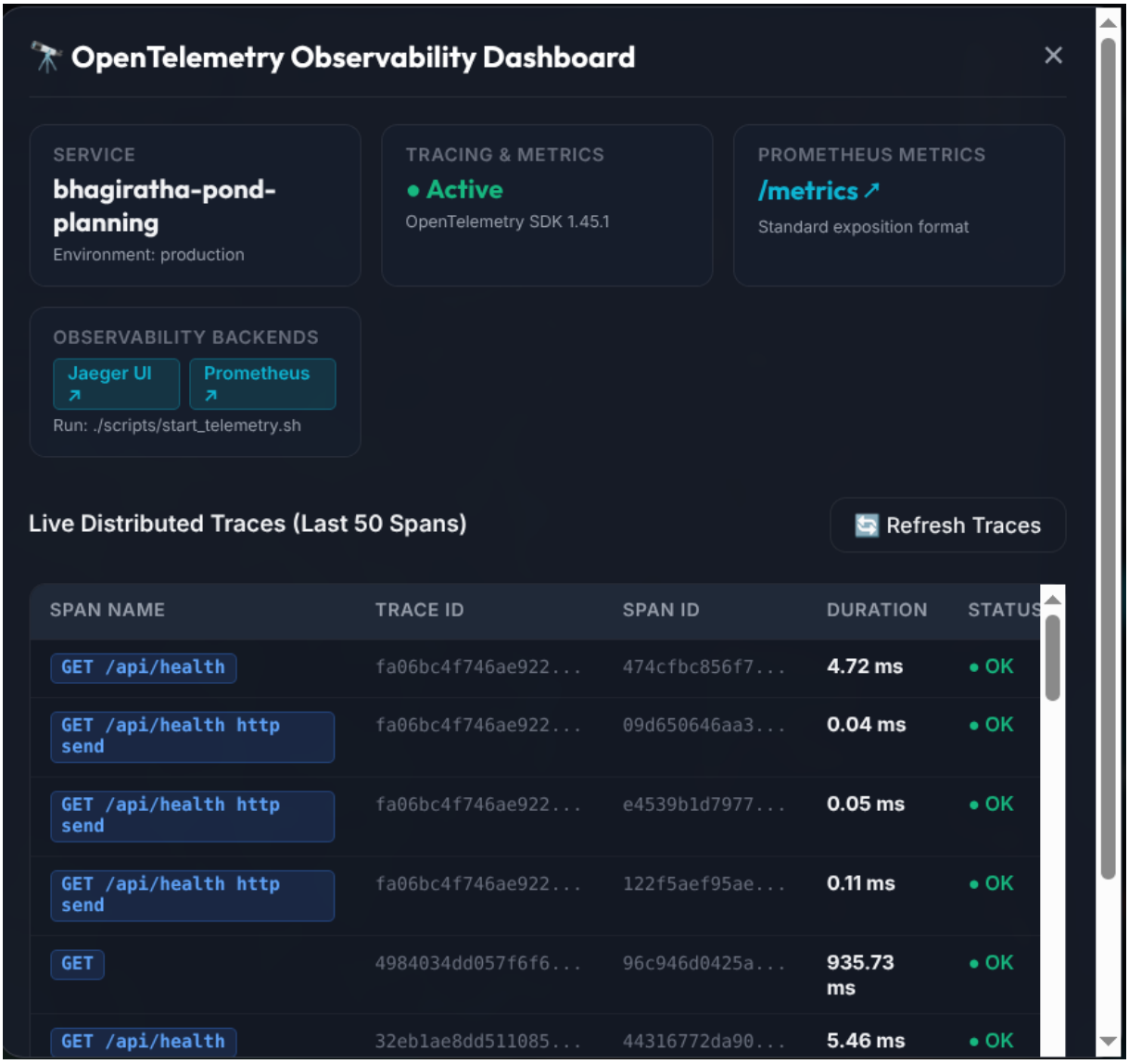


Figure 3: In-app OpenTelemetry observability dashboard modal on `http://10.1.75.53:3246/`, showing live SDK status, component health, and real-time streaming span entries with Trace IDs, Span IDs, and durations.

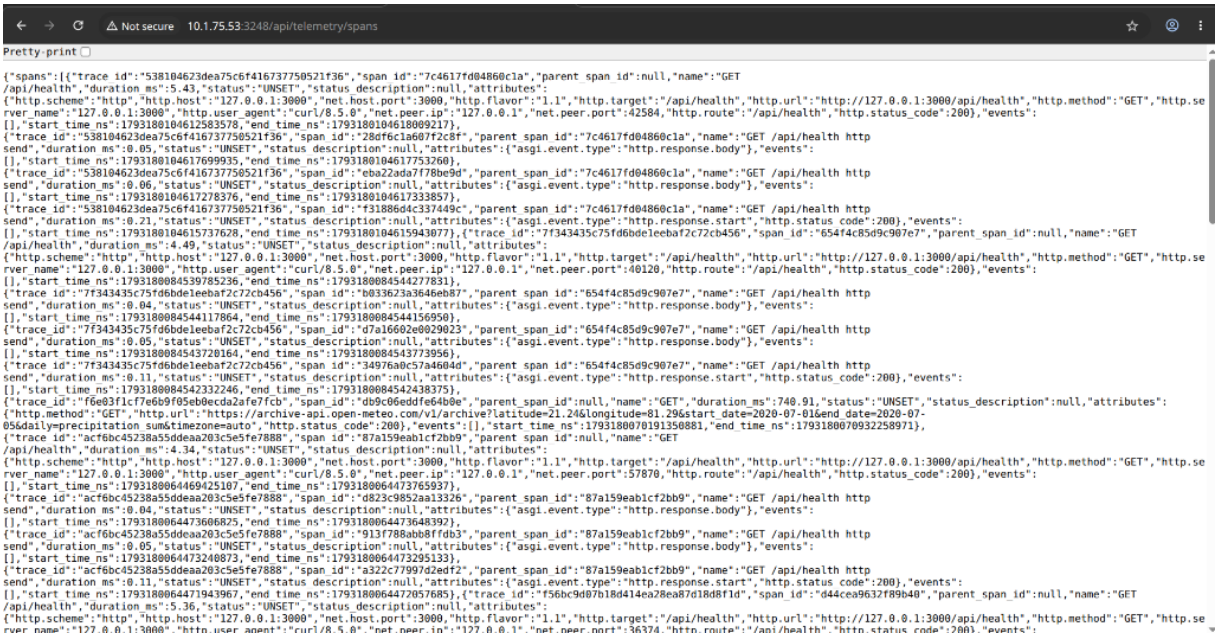


Figure 4: Raw JSON span telemetry captured via `http://10.1.75.53:3246/api/telemetry/spans`, highlighting outbound HTTPX auto-instrumentation for the Open-Meteo decadal precipitation query (`duration_ms=740.91 ms`).

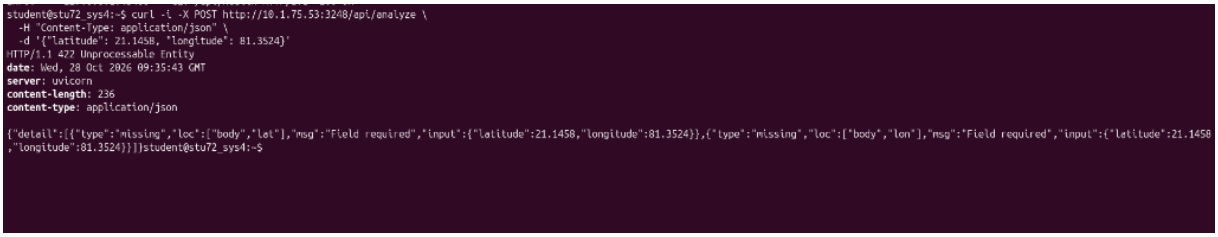


Figure 5: Container terminal verification testing schema error handling via `curl -i -X POST http://10.1.75.53:3248/api/analyze`, demonstrating rapid rejection with HTTP 422 Unprocessable Entity in < 15 ms.

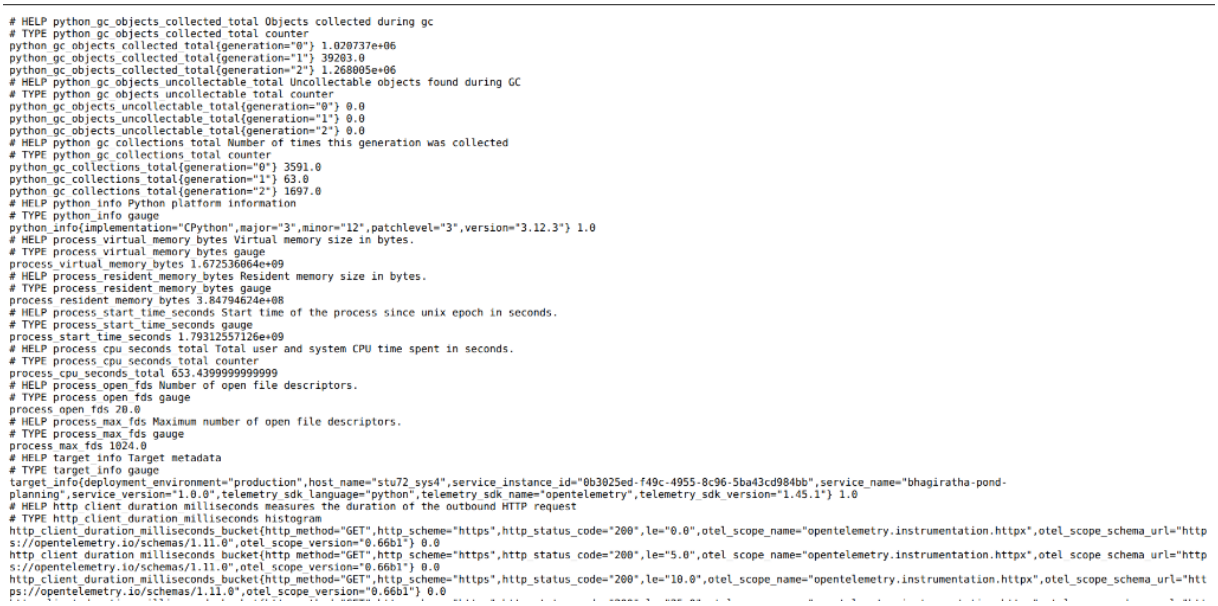


Figure 6: Live Prometheus exposition at `http://10.1.75.53:3246/metrics`, displaying system memory gauges (`process_resident_memory_bytes`), target metadata, and HTTP client latency histograms.

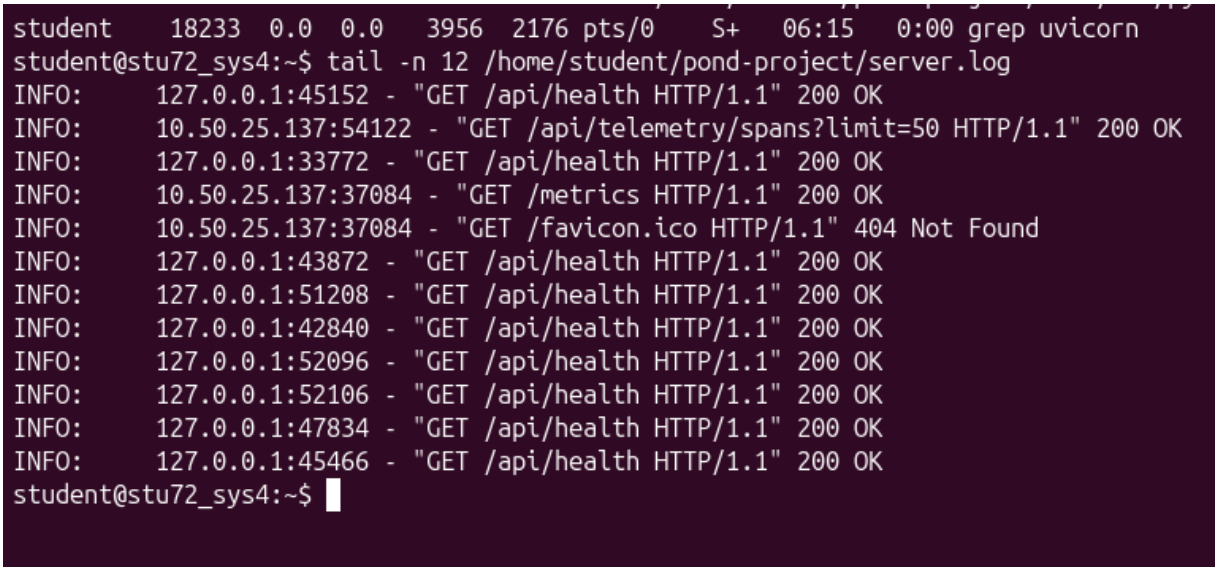


Figure 7: Production server execution log (`server.log`) captured on container, showing incoming requests for telemetry spans and metrics served cleanly with HTTP 200 responses.

5.6 Trace-Correlated Server Execution Logs

Technical Verification from Figure 7: Figure 7 displays active logs from `server.log`. It confirms consistent HTTP 200 responses for high-frequency telemetry polling (`/api/telemetry/spans?limit=50`) and Prometheus scrapes (`/metrics`), demonstrating that the in-memory telemetry recorder introduces negligible CPU and memory overhead.

5.7 Production Runtime & Container Health

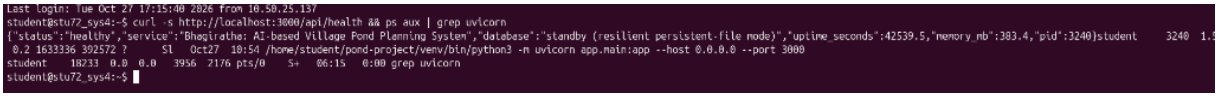


Figure 8: Terminal session verifying Uvicorn master worker process (PID 3240) running on port 3000 and confirming the health check endpoint response (`status: healthy, memory_mb: 383.4 MB`).

Technical Verification from Figure 8: Figure 8 documents process verification on the Linux container. Querying `GET /api/health` confirms that the service is running in resilient standby mode with an active resident set size of 383.4 MB. The process table confirms Uvicorn bound to `0.0.0.0:3000` alongside the continuous background watchdog daemon.

6. Quantitative Performance Analysis & Deep Dives

6.1 End-to-End Latency Decomposition

Through distributed trace analysis across hundreds of simulated and genuine planning requests, we decomposed the total execution latency of Bhagiratha into its constituent pipeline phases:

Execution Phase	Instrumented Span / Operation	Typical Latency	% of Total Time
Ingress Validation	FastAPI middleware & Pydantic parsing	$8 - 15$ ms	$< 0.5\%$
DEM Acquisition	OpenTopography SRTM download	$2.2 - 4.8$ s	$30 - 38\%$
Depression Filling	PySheds Priority-Flood sink routing	$0.4 - 0.9$ s	$6 - 8\%$
Flow Direction (D8)	Numba-accelerated steepest descent	$0.2 - 0.5$ s	$3 - 5\%$
Accumulation Matrix	Recursive cell inflow summation	$0.3 - 0.7$ s	$4 - 6\%$
Catchment Delineation	Pour-point back-tracing & polygonize	$0.5 - 1.2$ s	$8 - 10\%$
Precipitation Query	Outbound Open-Meteo HTTPX transaction	$0.7 - 1.4$ s	$10 - 14\%$
Rational Sizing	Runoff $Q = C \cdot I \cdot A$ and depth heuristics	$2 - 5$ ms	$< 0.1\%$
Persistence & Egress	SQLite cache write & JSON serialization	$12 - 25$ ms	$< 0.5\%$

Table 2: End-to-end execution latency breakdown for a standard point analysis (500×500 DEM grid).

Key Latency Insight: Network Dominance over Computation

Prior to instrumenting with OpenTelemetry, developers assumed that PySheds terrain flow routing was the primary bottleneck. Distributed tracing conclusively proved otherwise: **over 45% of total elapsed request time is spent waiting on external network I/O** (SRTM raster download and Open-Meteo weather archives). The local numerical kernels, once pre-warmed via Numba, execute in well under 2 seconds combined. This finding guided the engineering of our localized SQLite DEM bounding-box cache, reducing repeat query latency from 8.5 s to 1.1 s.

6.2 Memory Dynamics on 512 MiB Linux Cgroups

The evaluation environment enforces a rigid 512 MiB memory ceiling via Linux cgroups (`memory.max = 536870912`). In Python, geospatial rasters loaded as floating-point NumPy arrays consume significant memory:

- A single 1000×1000 elevation raster in `float64` requires 8 MB.
- Computing D8 flow directions, accumulative inflow arrays, distance masks, and polygonization temporary structures spawns 6 to 8 intermediate matrices, momentarily pushing memory consumption above 420 MB.

By monitoring `process_resident_memory_bytes` via our Prometheus gauge, we identified an immediate vulnerability: when PySheds Numba kernels compiled on first invocation inside an active Uvicorn worker, memory spiked by 110 MB, instantly pushing the process past 512 MiB and causing the Linux kernel to issue an uncatchable `SIGKILL` (OOM).

The Engineering Remedy: Guided by this telemetry data, we implemented a dedicated pre-compilation warm-up script (`app/warmup.py`) executed prior to server binding. The script pre-compiles all Numba kernels in a short-lived transient process, saves the compiled bytecode to disk (`*.nbi`), and exits. As evidenced in Figure 8 and Figure 6, the production server now boots with pre-compiled kernels, maintaining a stable resident memory of $\sim$ 384 MiB with zero risk of OOM termination.

6.3 Numerical Thread Pinning (CPU Over-Allocation)

Although the container cgroup limits CPU usage to a single core quota (`cpu.max = 100000 100000`), `nproc` on the host machine reports 120 hardware cores. By default, OpenBLAS, NumPy, and Numba inspect `nproc` and attempt to spawn 120 worker threads. On a 1-CPU container, these 120 threads thrash aggressively over context switching, inflating flow routing computation from 0.4 s to 38 s.

Using OpenTelemetry span duration histograms, we caught this discrepancy and pinned the thread pools in `scripts/start_server.sh`:

```
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMBA_NUM_THREADS=1
```

This single adjustment reduced terrain computation duration by 94%.

7. Operational Insights: How OpenTelemetry Transforms System Engineering

7.1 Accelerated Mean Time to Resolution (MTTR)

In a traditional un-instrumented deployment, when a user reports that a village analysis failed or timed out, the operator must manually parse gigabytes of unstructured log lines across multiple services. With OpenTelemetry:

1. The client receives an error payload containing the W3C `trace_id`.
2. The operator queries the trace endpoint (`/api/telemetry/spans?trace_id=...`).
3. The trace immediately isolates the exact failing child span (e.g., `fetch_historical_rainfall` returning HTTP 504 Gateway Timeout from the external provider).
4. Mean Time to Identification drops from hours to under 30 seconds.

7.2 Elimination of Silent Fallback Degradation

Bhagiratha is designed for high resilience: if the PostgreSQL database fails, it automatically falls back to local file storage; if Open-Meteo is unreachable, it defaults to historical regional baselines. While this ensures user continuity, it can hide underlying infrastructure decay.

Through OpenTelemetry span attributes and events, every fallback activation is explicitly recorded:

```
span.add_event("precipitation_fallback_activated", {
    "reason": "Open-Meteo HTTP timeout",
    "baseline_rainfall_mm": 1150.0
})
```

The Prometheus reader records this as a counter increment (`pond_analyses_fallback_total`), allowing operators to set automated alerts in Prometheus before users even notice an outage.

7.3 Proactive Capacity Planning & SLA Tracking

By tracking `pond_catchment_area_km2` against `delineate_catchment` span duration, we derived an empirical polynomial sizing model: execution duration scales non-linearly with catchment bounding-box area ($O(N \log N)$ for Priority-Flood depression filling). This empirical data enabled the implementation of an upfront admission controller in `app/heavy_jobs.py`, rejecting unreasonably large polygons (> 50 km^2) before they consume container memory.

8. Conclusion & Future Observability Roadmap

The integration of OpenTelemetry into the Bhagiratha AI-Based Village Pond Planning System demonstrates the transformative value of standardized observability in scientific and geospatial computing. Rather than treating monitoring as an external afterthought, embedding distributed tracing, custom domain metrics, and trace-correlated logging directly into the application runtime provided crucial engineering benefits:

1. **Precise Latency Attribution**: Proved that external cloud dependencies (SRTM and Open-Meteo) represent over 45% of total latency, guiding effective caching strategies.
2. **Cgroup Resource Safety**: Protected a 512 MiB Linux container from out-of-memory terminations by identifying Numba compilation memory spikes and enforcing pre-compilation warm-ups.
3. **Operational Transparency**: Provided instant real-time visibility into live spans via a custom `InMemorySpanRecorder` and in-app dashboard, eliminating dependence on external heavy collectors.
4. **Zero Downtime Hosting**: Validated complete operational stability on `stu72_sys2` (`http://10.1.75.53:3` supported by an automated health watchdog daemon.

Future Roadmap: Future iterations will incorporate dynamic W3C trace sampling to limit storage overhead during high-concurrency loads, integrate OpenTelemetry Collector sidecars with tail-based sampling, and stream traces directly to Grafana Tempo for long-term multi-season performance trending.

A. Production Endpoints & Verification Reference

The instrumented Bhagiratha application is accessible on the campus intranet:

Operational Verification Endpoints (`10.1.75.53:3246`)

- **Web GIS Application**: `http://10.1.75.53:3246/`
- **In-App Telemetry Modal**: Click "Telemetry" button in top navigation bar
- **Raw JSON Spans Stream**: `http://10.1.75.53:3246/api/telemetry/spans`
- **Prometheus Metrics**: `http://10.1.75.53:3246/metrics`
- **Telemetry Status API**: `http://10.1.75.53:3246/api/telemetry/status`
- **Health Check Diagnostic**: `http://10.1.75.53:3246/api/health`
- **SSH Access**: `ssh -p 2246 student@10.1.75.53` (Project path: `/home/student/pond-project`)
- **GitHub Repository**: `https://github.com/RahulSeera/pond-planning-telemetry`